K8s Pod Visualizer — Manual de Instalação v3.0.1

CentralDevOps · Versão do documento: 3.0.1 · Data: Março 2026

Índice

1. Visão Geral
2. Pré-requisitos
3. Comparativo das Modalidades de Instalação
4. Instalação Rápida com Script Automático
5. Instalação Manual — Sem Persistência
6. Instalação Manual — Com Persistência
7. Instalação por Ambiente Kubernetes
8. Configuração de Autenticação SRE / Squad
9. Configuração Avançada
10. Atualização de Versão Anterior
11. Verificação Pós-Instalação
12. Solução de Problemas
13. Desinstalação
14. Referência de Variáveis de Ambiente
15. Referência de RBAC

---

1. Visão Geral

O **K8s Pod Visualizer** é um painel de monitoramento em tempo real para clusters Kubernetes. Ele exibe pods como bolhas animadas agrupadas por namespace, com métricas de CPU e memória, monitoramento de deployments, planejamento de capacidade e, a partir da v3.0, autenticação com dois perfis de acesso distintos: **SRE** (acesso total ao cluster) e **Squad** (acesso restrito ao próprio namespace).

A v3.0.1 corrige um erro crítico de inicialização (`ERR_MODULE_NOT_FOUND: auth.js`) presente na v3.0.0 e é a versão recomendada para produção.

Arquitetura

```
┌──────────────────────────────────────────────────────┐
│                    Pod do Visualizer                 │
│                                                      │
│  ┌──────────────┐    ┌──────────────────────────┐   │
│  │  Frontend    │    │  Backend Node.js         │   │
│  │  React/Vite  │◄──►│  server-in-cluster.js    │   │
│  │  (estático)  │    │  auth.js (JWT/bcrypt)    │   │
│  └──────────────┘    │  db.js (SQLite opcional) │   │
│                      └──────────────────────────┘   │
└──────────────────────────────┬──────────────────────┘
                               │ ServiceAccount
                               ▼
                   ┌──────────────────────┐
                   │  Kubernetes API Server  │
                   └──────────────────────┘
```

O servidor Node.js roda **dentro do cluster** e acessa a API do Kubernetes via `ServiceAccount` com token automático, sem necessidade de `kubeconfig` externo.

---

2. Pré-requisitos

Ferramentas necessárias

Ferramenta	Versão mínima	Verificação
`kubectl`	1.24+	`kubectl version --client`
`docker`	20.10+	`docker --version`
`git`	2.x	`git --version`
`openssl`	1.1+	`openssl version`

Requisitos do cluster

Requisito	Obrigatório	Observação
Kubernetes	1.24+	Testado até 1.31
Acesso `cluster-admin`	Sim	Para criar ClusterRole e ServiceAccount
Metrics Server	Não	Sem ele, Capacity Planning usa dados simulados
StorageClass disponível	Apenas com persistência	Qualquer StorageClass funciona
Ingress Controller	Não	Alternativa ao NodePort

Verificar acesso ao cluster

```
kubectl cluster-info
kubectl auth can-i create clusterrolebindings --all-namespaces
```

---

3. Comparativo das Modalidades de Instalação

Característica	Sem Persistência	Com Persistência
Banco de dados	Nenhum	SQLite em PVC
Histórico de eventos	Não	Sim (30 dias)
Histórico de deployments	Não	Sim (60 dias)
Gráfico de tendência 24h	Não	Sim
Snapshots de capacidade	Não	Sim (a cada 5 min)
Reinicializações	Sem perda	Dados preservados
PVC necessário	Não	Sim (1Gi padrão)
Complexidade	Baixa	Média
Recomendado para	Avaliação, ambientes efêmeros	Produção

---

4. Instalação Rápida com Script Automático

O script de instalação automatiza todas as etapas: criação do namespace, RBAC, Secret JWT, Deployment e Service.

Sem persistência

```
# Baixar o script
curl -sL https://raw.githubusercontent.com/divinoandersonbastos/k8s-pod-visualizer/main/k8s/install-no-persistence.sh \
  -o install.sh && chmod +x install.sh

# Executar (substitua os valores conforme seu ambiente)
./install.sh \
  --cluster-name "meu-cluster" \
  --namespace "k8s-pod-visualizer" \
  --nodeport 30080
```

Com persistência

```
# Baixar o script
curl -sL https://raw.githubusercontent.com/divinoandersonbastos/k8s-pod-visualizer/main/k8s/install-with-persistence.sh \
  -o install.sh && chmod +x install.sh

# Executar
./install.sh \
  --cluster-name "meu-cluster" \
  --namespace "k8s-pod-visualizer" \
  --nodeport 30080 \
  --storage-class "standard" \
  --storage-size "2Gi"
```

Parâmetros disponíveis

Parâmetro	Padrão	Descrição
`--cluster-name`	`kubernetes`	Nome exibido no header do painel
`--namespace`	`k8s-pod-visualizer`	Namespace de instalação
`--nodeport`	`30080`	Porta NodePort de acesso
`--jwt-secret`	Gerado automaticamente	Secret JWT (64 chars hex)
`--jwt-expires`	`8h`	Tempo de expiração do token
`--storage-class`	Detectado automaticamente	StorageClass do PVC (apenas com persistência)
`--storage-size`	`1Gi`	Tamanho do PVC (apenas com persistência)
`--dry-run`	—	Exibe o que seria aplicado sem executar
`--uninstall`	—	Remove todos os recursos criados

---

5. Instalação Manual — Sem Persistência

Esta modalidade não cria PVC nem banco de dados. Ideal para avaliação ou ambientes onde a persistência é gerenciada externamente.

Passo 1 — Clonar o repositório

```
git clone https://github.com/divinoandersonbastos/k8s-pod-visualizer.git
cd k8s-pod-visualizer
```

Passo 2 — Construir a imagem Docker

```
docker build -t k8s-pod-visualizer:3.0.1 .
```

Se estiver usando um registry privado:

```
docker tag k8s-pod-visualizer:3.0.1 SEU_REGISTRY/k8s-pod-visualizer:3.0.1
docker push SEU_REGISTRY/k8s-pod-visualizer:3.0.1
```

Passo 3 — Gerar o JWT Secret

```
JWT_SECRET=$(openssl rand -hex 32)
echo "JWT_SECRET gerado: $JWT_SECRET"
# Guarde esse valor em local seguro
```

Passo 4 — Aplicar o manifest

```
# Editar o manifest para ajustar imagem e JWT_SECRET
cp k8s/deploy-no-persistence.yaml k8s/deploy-custom.yaml
```

Abra `k8s/deploy-custom.yaml` e substitua os placeholders:

```yaml
# Linha do Secret — substituir o valor base64
data:
  jwt-secret: <SUBSTITUA_PELO_BASE64_DO_JWT_SECRET>
  # Para gerar: echo -n "SEU_JWT_SECRET" | base64

# Linha da imagem — substituir pela sua imagem
image: SEU_REGISTRY/k8s-pod-visualizer:3.0.1

# Nome do cluster
- name: CLUSTER_NAME
  value: "nome-do-seu-cluster"
```

Gerar o valor base64 do Secret:

```bash
echo -n "$JWT_SECRET" | base64
```

Aplicar:

```bash
kubectl apply -f k8s/deploy-custom.yaml
```

Passo 5 — Verificar a instalação

```bash
kubectl get all -n k8s-pod-visualizer
kubectl rollout status deployment/k8s-pod-visualizer -n k8s-pod-visualizer
```

Passo 6 — Acessar o painel

```bash
# Obter o IP de um node
NODE_IP=$(kubectl get nodes -o jsonpath='{.items[0].status.addresses[?
(@.type=="InternalIP")].address}')
echo "Acesse: http://$NODE_IP:30080"
```

---

6. Instalação Manual — Com Persistência

Esta modalidade cria um PVC para armazenar o banco SQLite, habilitando histórico de eventos, gráficos de tendência e snapshots de capacidade.

Passo 1 — Clonar e construir

Siga os **Passos 1 e 2** da seção anterior (Instalação Manual — Sem Persistência).

Passo 2 — Verificar StorageClass disponível

```
kubectl get storageclass
```

Anote o nome da StorageClass que será usada (ex: `standard`, `gp2`, `managed-premium`).

Passo 3 — Gerar o JWT Secret

```
JWT_SECRET=$(openssl rand -hex 32)
JWT_SECRET_B64=$(echo -n "$JWT_SECRET" | base64)
echo "JWT_SECRET_B64: $JWT_SECRET_B64"
```

Passo 4 — Configurar e aplicar o manifest

```
cp k8s/deploy-with-persistence.yaml k8s/deploy-custom.yaml
```

Edite `k8s/deploy-custom.yaml` e substitua:

```yaml
# Secret JWT
data:
  jwt-secret: <SUBSTITUA_PELO_JWT_SECRET_BASE64>

# Imagem
image: SEU_REGISTRY/k8s-pod-visualizer:3.0.1

# StorageClass do PVC
storageClassName: standard   # substitua pelo nome da sua StorageClass

# Tamanho do PVC (ajuste conforme necessidade)
storage: 1Gi

# Nome do cluster
- name: CLUSTER_NAME
  value: "nome-do-seu-cluster"
```

Aplicar:

```
kubectl apply -f k8s/deploy-custom.yaml
```

Passo 5 — Verificar PVC e Pod

```
# PVC deve estar Bound
kubectl get pvc -n k8s-pod-visualizer

# Pod deve estar Running
kubectl get pods -n k8s-pod-visualizer

# Verificar logs do banco
kubectl logs deployment/k8s-pod-visualizer -n k8s-pod-visualizer | grep -i
"sqlite\|database\|migration"
```

Saída esperada nos logs:

```
[DB] SQLite inicializado em /app/data/events.db
[DB] Migração v1 aplicada
[DB] Migração v2 aplicada
[DB] Migração v3 aplicada
[DB] Migração v4 (users) aplicada
```

---

7. Instalação por Ambiente Kubernetes

GKE (Google Kubernetes Engine)

```
# Autenticar
gcloud container clusters get-credentials NOME_DO_CLUSTER \
  --region REGIAO --project PROJETO

# Usar StorageClass padrão do GKE
./install.sh --cluster-name "gke-producao" --storage-class "standard-rwo"
```

EKS (Amazon Elastic Kubernetes Service)

```
# Autenticar
aws eks update-kubeconfig --name NOME_DO_CLUSTER --region REGIAO

# Usar StorageClass do EBS
./install.sh --cluster-name "eks-producao" --storage-class "gp2"
```

> ***Atenção EKS:*** *O metrics-server não vem instalado por padrão. Para habilitar o Capacity Planning com dados reais:*

> ```
> kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
> ```
>
> ### AKS (Azure Kubernetes Service)
>
> ```bash
> # Autenticar
> az aks get-credentials --resource-group GRUPO --name NOME_DO_CLUSTER
>
> # Usar StorageClass padrão do AKS
> ./install.sh --cluster-name "aks-producao" --storage-class "managed-premium"
> ```

k3s (On-Premise / Edge)

```
# k3s já inclui metrics-server e StorageClass local-path
./install.sh --cluster-name "k3s-producao" --storage-class "local-path"
```

Kubernetes On-Premise (kubeadm)

```
# Verificar StorageClass disponível
kubectl get storageclass

# Se não houver StorageClass, usar sem persistência
./install-no-persistence.sh --cluster-name "on-premise"

# Ou instalar o NFS Provisioner para habilitar PVC
helm repo add nfs-subdir-external-provisioner \
  https://kubernetes-sigs.github.io/nfs-subdir-external-provisioner/
helm install nfs-provisioner nfs-subdir-external-provisioner/nfs-subdir-external-provisioner \
  --set nfs.server=IP_DO_NFS --set nfs.path=/exports
```

Rancher / RKE2

```
# Usar StorageClass do Longhorn (se instalado)
./install.sh --cluster-name "rancher-producao" --storage-class "longhorn"
```

---

8. Configuração de Autenticação SRE / Squad

A v3.0 introduz autenticação JWT com dois perfis de acesso. A autenticação é **opcional**: se o backend não estiver disponível ou o `JWT_SECRET` não estiver configurado, o painel funciona em modo aberto (sem login).

Perfis de Acesso

Funcionalidade	SRE	Squad
Visualização de todos os namespaces	✅	❌ (apenas os autorizados)
Deploy Monitor	✅	✅ (apenas seu namespace)
Capacity Planning	✅	❌
Node Monitor	✅	❌
Editor de Recursos YAML	✅	❌
Gestão de usuários Squad	✅	❌
Painel de Trace (Jaeger/Tempo)	✅	✅ (apenas seu namespace)
Personalização visual	✅	✅

Primeiro acesso — criar usuário SRE

1. Acesse o painel no navegador (`http://NODE_IP:30080`)
2. Na tela de login, clique em **“Novo usuário”**
3. Preencha o nome de usuário e uma senha forte (mínimo 8 caracteres)
4. Clique em **“Criar conta SRE”**

5. Faça login com as credenciais criadas

> *O primeiro usuário criado é sempre SRE. Usuários subsequentes criados pelo SRE são do perfil Squad.*

Criar usuários Squad

Após login como SRE:

1. Clique no ícone de **Usuários** (ícone de pessoas) no header
2. Clique em **"Novo usuário Squad"**
3. Preencha nome, senha e selecione os **namespaces autorizados**
4. Clique em **"Criar"**

O usuário Squad verá apenas os pods, deployments e traces dos namespaces autorizados.

Rotação do JWT Secret

Para rotacionar o `JWT_SECRET` (invalida todas as sessões ativas):

```
# Gerar novo secret
NEW_SECRET=$(openssl rand -hex 32)
NEW_SECRET_B64=$(echo -n "$NEW_SECRET" | base64)

# Atualizar o Secret no Kubernetes
kubectl patch secret k8s-pod-visualizer-secrets \
  -n k8s-pod-visualizer \
  --type='json' \
  -p="[{\"op\":\"replace\",\"path\":\"/data/jwt-
secret\",\"value\":\"$NEW_SECRET_B64\"}]"

# Reiniciar o pod para aplicar o novo secret
kubectl rollout restart deployment/k8s-pod-visualizer -n k8s-pod-visualizer
```

> *Após a rotação, todos os usuários precisarão fazer login novamente.*

---

9. Configuração Avançada

Configurar Ingress (alternativa ao NodePort)

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: k8s-pod-visualizer
  namespace: k8s-pod-visualizer
  annotations:
    nginx.ingress.kubernetes.io/proxy-read-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "3600"
spec:
  ingressClassName: nginx
  rules:
    - host: k8s-visualizer.meudominio.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: k8s-pod-visualizer
                port:
                  number: 3000
```

```
kubectl apply -f ingress.yaml
```

Configurar Trace (Jaeger / Grafana Tempo)

No painel, como SRE:

1. Clique no ícone de **Trace** no header
2. Selecione o provedor: **Jaeger** ou **Grafana Tempo**
3. Informe a URL base (ex: `http://jaeger-query.observability:16686`)
4. Associe namespaces aos serviços de trace

Para Jaeger com autenticação:

```
# Adicionar credenciais como Secret
kubectl create secret generic k8s-pod-visualizer-trace \
  --from-literal=jaeger-url="http://jaeger-query.observability:16686" \
  --from-literal=jaeger-token="SEU_TOKEN" \
  -n k8s-pod-visualizer
```

Configurar limite de headroom (Capacity Planning)

No painel, clique em **Configurações** (ícone de engrenagem) e ajuste o slider de **Headroom mínimo** (padrão: 20%). Um alerta visual será exibido quando qualquer node-pool ultrapassar `100% - headroom`.

Ajustar intervalo de polling

```
# No Deployment, adicionar variável de ambiente
env:
  - name: POLL_INTERVAL_MS
    value: "15000"   # padrão: 10000 (10 segundos)
  - name: CAPACITY_SNAPSHOT_INTERVAL_MS
    value: "300000"  # padrão: 300000 (5 minutos)
```

---

10. Atualização de Versão Anterior

De v2.x para v3.0.1

A v3.0 adiciona autenticação JWT e novas tabelas no SQLite. A migração do banco é automática.

```
# 1. Atualizar o código
cd /opt/k8s-pod-visualizer
git pull origin main

# 2. Rebuildar a imagem
docker build -t k8s-pod-visualizer:3.0.1 .
docker tag k8s-pod-visualizer:3.0.1 SEU_REGISTRY/k8s-pod-visualizer:3.0.1
docker push SEU_REGISTRY/k8s-pod-visualizer:3.0.1

# 3. Criar o Secret JWT (novo na v3.0)
JWT_SECRET=$(openssl rand -hex 32)
kubectl create secret generic k8s-pod-visualizer-secrets \
  --from-literal=jwt-secret="$JWT_SECRET" \
  -n k8s-pod-visualizer

# 4. Adicionar variáveis de ambiente ao Deployment existente
kubectl set env deployment/k8s-pod-visualizer \
  JWT_SECRET="$JWT_SECRET" \
  JWT_EXPIRES_IN="8h" \
  -n k8s-pod-visualizer

# 5. Atualizar a imagem
kubectl set image deployment/k8s-pod-visualizer \
  k8s-pod-visualizer=SEU_REGISTRY/k8s-pod-visualizer:3.0.1 \
  -n k8s-pod-visualizer

# 6. Aguardar o rollout
kubectl rollout status deployment/k8s-pod-visualizer -n k8s-pod-visualizer
```

> ***Nota:*** *Se você estava usando a v3.0.0 (com o bug do* `auth.js`*), a v3.0.1 corrige o problema. Basta rebuildar a imagem e atualizar o Deployment — não é necessário recriar o Secret JWT.*

De v1.x para v3.0.1

Recomenda-se uma instalação limpa:

```
# Remover instalação antiga (preserva o PVC se existir)
kubectl delete
deployment,service,serviceaccount,clusterrole,clusterrolebinding \
  -l app=k8s-pod-visualizer -n k8s-pod-visualizer

# Instalar a v3.0.1
./install.sh --cluster-name "meu-cluster"
```

---

11. Verificação Pós-Instalação

Execute esta sequência após qualquer instalação ou atualização:

```
# 1. Verificar status dos recursos
kubectl get all -n k8s-pod-visualizer

# 2. Verificar se o pod está Running (não CrashLoopBackOff)
kubectl get pods -n k8s-pod-visualizer -w

# 3. Verificar logs de inicialização
kubectl logs deployment/k8s-pod-visualizer -n k8s-pod-visualizer --tail=50

# 4. Verificar acesso à API do Kubernetes (RBAC)
kubectl logs deployment/k8s-pod-visualizer -n k8s-pod-visualizer | grep -i
"error\|forbidden\|unauthorized"

# 5. Testar o endpoint de saúde
NODE_IP=$(kubectl get nodes -o jsonpath='{.items[0].status.addresses[?
(@.type=="InternalIP")].address}')
curl -s http://$NODE_IP:30080/api/cluster-info | python3 -m json.tool

# 6. Verificar o banco de dados (apenas com persistência)
kubectl exec deployment/k8s-pod-visualizer -n k8s-pod-visualizer -- \
  ls -lh /app/data/
```

Saída esperada nos logs

```
[SERVER] K8s Pod Visualizer v3.0.1 iniciando...
[DB] SQLite inicializado em /app/data/events.db
[DB] Migração v4 (users) aplicada
[AUTH] Sistema de autenticação JWT inicializado
[SERVER] Servidor HTTP escutando na porta 3000
[K8S] Conectado ao cluster: https://10.96.0.1
[K8S] Namespace padrão: k8s-pod-visualizer
```

---

12. Solução de Problemas

`ERR_MODULE_NOT_FOUND: Cannot find module '/app/auth.js'`

Causa: Imagem construída com versão anterior do Dockerfile (bug da v3.0.0).

Solução:

```
git pull origin main   # garante o Dockerfile corrigido
docker build -t k8s-pod-visualizer:3.0.1 .
kubectl set image deployment/k8s-pod-visualizer \
  k8s-pod-visualizer=k8s-pod-visualizer:3.0.1 \
  -n k8s-pod-visualizer
```

`SyntaxError: Unexpected identifier` ao iniciar

Causa: Arquivo JavaScript corrompido na imagem.

Solução:

```
# Verificar sintaxe localmente
node --check server-in-cluster.js
node --check auth.js
node --check db.js

# Rebuildar a imagem após corrigir
docker build --no-cache -t k8s-pod-visualizer:3.0.1 .
```

Pod em `CrashLoopBackOff`

```
# Ver o motivo do crash
kubectl describe pod -l app=k8s-pod-visualizer -n k8s-pod-visualizer
kubectl logs -l app=k8s-pod-visualizer -n k8s-pod-visualizer --previous
```

Causas comuns:

Mensagem de erro	Causa	Solução
`ERR_MODULE_NOT_FOUND: auth.js`	Dockerfile desatualizado	Rebuildar com v3.0.1
`SQLITE_CANTOPEN`	PVC sem permissão de escrita	Verificar `chmod 777 /app/data`
`EACCES: permission denied`	SecurityContext restritivo	Adicionar `runAsUser: 1000`
`Forbidden` na API K8s	RBAC insuficiente	Aplicar ClusterRole atualizado

---

`403 Forbidden` ao acessar a API do Kubernetes

```
# Verificar o ClusterRoleBinding
kubectl get clusterrolebinding k8s-pod-visualizer -o yaml

# Testar permissões do ServiceAccount
kubectl auth can-i list pods \
  --as=system:serviceaccount:k8s-pod-visualizer:k8s-pod-visualizer \
  --all-namespaces
```

Se retornar `no`, reaplicar o ClusterRole:

```
kubectl apply -f k8s/deploy-no-persistence.yaml  # ou deploy-with-persistence.yaml
```

PVC em `Pending` (apenas com persistência)

```
kubectl describe pvc k8s-pod-visualizer-data -n k8s-pod-visualizer
```

Causas comuns:

- **StorageClass não existe:** `kubectl get storageclass` e ajustar o manifest
- **Sem nodes com capacidade:** verificar `kubectl describe nodes`
- **Provisioner não instalado:** instalar o provisioner da StorageClass

---

Painel abre mas não mostra pods

1. Verificar se o metrics-server está instalado: `kubectl top pods -A`
2. Verificar logs de erro: `kubectl logs deployment/k8s-pod-visualizer -n k8s-pod-visualizer | grep ERROR`
3. Verificar RBAC: `kubectl auth can-i list pods --as=system:serviceaccount:k8s-pod-visualizer:k8s-pod-visualizer -A`

Tela de login não aparece (modo sem autenticação)

O painel opera em modo aberto quando:

- O `JWT_SECRET` não está configurado como variável de ambiente
- O backend não consegue inicializar o módulo de autenticação

Para forçar o modo autenticado, confirme que o Secret está montado:

```
kubectl exec deployment/k8s-pod-visualizer -n k8s-pod-visualizer -- \
  printenv JWT_SECRET
```

---

13. Desinstalação

Remoção completa (apaga todos os dados)

```
# Usando o script
./install.sh --uninstall

# Ou manualmente
kubectl delete namespace k8s-pod-visualizer
kubectl delete clusterrole k8s-pod-visualizer
kubectl delete clusterrolebinding k8s-pod-visualizer
```

Remoção preservando dados (PVC)

```
# Remover apenas o Deployment e Service (preserva PVC e namespace)
kubectl delete deployment,service k8s-pod-visualizer -n k8s-pod-visualizer

# Para restaurar depois
kubectl apply -f k8s/deploy-with-persistence.yaml
```

---

14. Referência de Variáveis de Ambiente

Variável	Padrão	Descrição
`PORT`	`3000`	Porta do servidor HTTP
`NODE_ENV`	`production`	Ambiente Node.js
`K8S_API_URL`	`https://kubernetes.default.svc`	URL da API do Kubernetes
`USE_SERVICE_ACCOUNT`	`true`	Usar token do ServiceAccount
`CLUSTER_NAME`	`kubernetes`	Nome exibido no header
`DATA_DIR`	`/app/data`	Diretório do banco SQLite
`DISABLE_DB`	`false`	Desabilitar SQLite completamente
`JWT_SECRET`	—	**Obrigatório para autenticação.** Mínimo 32 chars
`JWT_EXPIRES_IN`	`8h`	Expiração do token JWT
`POLL_INTERVAL_MS`	`10000`	Intervalo de polling de pods (ms)
`CAPACITY_SNAPSHOT_INTERVAL_MS`	`300000`	Intervalo de snapshots de capacidade (ms)
`EVENT_RETENTION_DAYS`	`30`	Retenção de eventos de pods

Variável	Padrão	Descrição
		no SQLite
`DEPLOY_EVENT_RETENTION_DAYS`	`60`	Retenção de eventos de deployments
`CAPACITY_RETENTION_DAYS`	`3`	Retenção de snapshots de capacidade

---

15. Referência de RBAC

O ClusterRole abaixo lista todas as permissões necessárias para a v3.0.1, incluindo as permissões de escrita do Editor de Recursos (exclusivas para SRE).

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: k8s-pod-visualizer
rules:
  # Pods — leitura e métricas
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
  - apiGroups: ["metrics.k8s.io"]
    resources: ["pods", "nodes"]
    verbs: ["get", "list"]

  # Nodes — leitura
  - apiGroups: [""]
    resources: ["nodes"]
    verbs: ["get", "list", "watch"]

  # Namespaces e Events — leitura
  - apiGroups: [""]
    resources: ["namespaces", "events"]
    verbs: ["get", "list", "watch"]

  # ConfigMaps — leitura e escrita (Editor de Recursos)
  - apiGroups: [""]
    resources: ["configmaps"]
    verbs: ["get", "list", "watch", "patch", "update"]

  # Deployments e ReplicaSets — leitura e escrita
  - apiGroups: ["apps"]
    resources: ["deployments", "replicasets", "daemonsets", "statefulsets"]
    verbs: ["get", "list", "watch", "patch", "update"]

  # HPA — leitura e escrita
  - apiGroups: ["autoscaling"]
    resources: ["horizontalpodautoscalers"]
    verbs: ["get", "list", "watch", "patch", "update"]
```

> ***Nota de segurança:*** *Se você não quiser habilitar o Editor de Recursos (modo somente leitura), remova os verbos* `patch` *e* `update` *de todas as regras. O painel continuará funcionando normalmente — apenas o botão de edição ficará desabilitado.*

Manual gerado para K8s Pod Visualizer v3.0.1 — CentralDevOps